SecEBL: A Behavior-Intent Layer for Intrusion Detection

Abstract

What is intrusion detection trying to do?

It is not merely a matter of collecting more rules or memorizing more bad strings. Intrusion detection must answer a more basic question: can the system **understand the visible behavioral intent in an event**, and can it determine whether those behaviors are forming an attack chain in context?

Detection engineering has long tried to bridge this gap in intent understanding. Allowlists, denylists, regular expressions, IOC intelligence, YARA and Sigma rules, statistical anomaly detection, risk scoring, and manual tuning all approximate the same objective: helping machines infer what is happening from raw telemetry. These methods are effective and will remain part of detection systems. Their limitation is that they often depend on known forms: a string, path, hash, domain, parameter combination, threshold, or historical baseline. Once an attacker changes syntax, uses a legitimate tool, reorders arguments, splits a behavior across several commands, or maps the same Linux, Kubernetes, or cloud semantics onto a different telemetry surface, detection often falls back into rule iteration, false-positive suppression, exception handling, and policy patching.

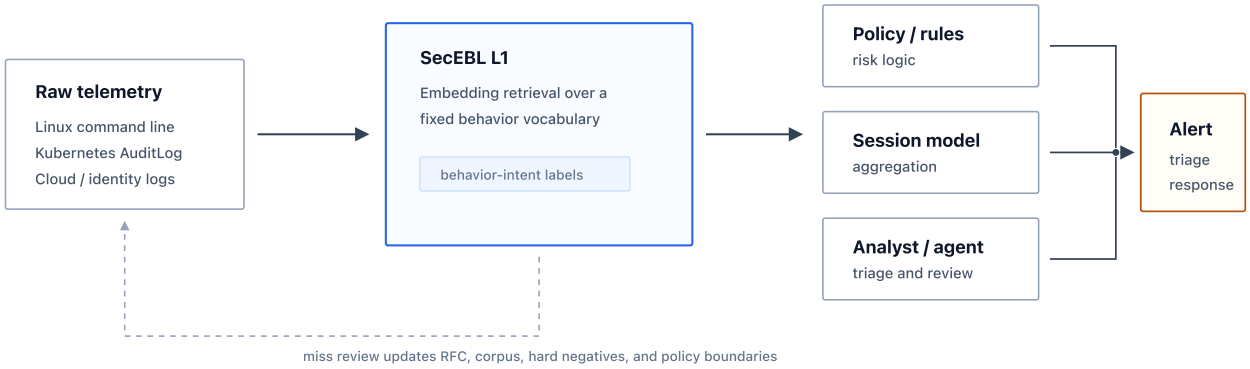
AI-assisted offensive workflows make this problem more visible. Attack chains, payload variants, LOTL command sequences, cloud and Kubernetes operation scripts, obfuscated text, and bypass samples can now be produced at lower cost and higher speed. If defense continues to rely mainly on rule-by-rule patches, case-by-case false-positive cleanup, and tool-specific argument tracking, it will remain structurally slow. The goal should not be a black-box model that claims to replace rules. The objective is a layer that first converts telemetry into interpretable behavioral semantics: the system should first answer "what behavior is visible in this event", and then allow IOC matching, rules, policy, session models, or analysts to decide risk.

SecEBL, the Security Event Behavior Labeler, studies this problem directly: whether raw security events can be converted into **explicit, interpretable, composable behavior-intent evidence** so that detection engineering can operate on a behavioral semantic layer rather than only on strings and rule hits.

SecEBL is not designed to replace IOCs, allowlists, denylists, or rules with embedding. Instead, it places embedding in the middle of the detection pipeline:

```
raw security event
  -> behavior-intent labels
  -> session / policy / analyst reasoning
  -> alert, triage, response
```

Figure 1. SecEBL in the detection pipeline



More precisely, SecEBL attempts to move the basic detection unit from "whether a string matched" to "whether a behavior is present". Regular expressions, IOCs, and rules remain foundational security engineering tools. SecEBL adds a behavioral semantic primitive alongside them: composable, auditable, and suitable for existing pipelines, but matching against visible behaviors such as `read_credential_material`, `spawn_reverse_shell`, `execute_in_workload`, `grant_cloud_privilege`, and `query_service_health` rather than only text fragments.

SecEBL has been validated on Linux command-line telemetry and normalized Kubernetes AuditLog telemetry. It consolidates security behavior into 12 behavior groups, 361 behavior-intent labels, and a 1,727-line tagging RFC. It uses embedding retrieval to map raw events into a fixed behavior vocabulary rather than allowing the model to generate free-form judgments.

The public release includes GitHub code, schema, the tagging RFC, public examples, evaluation scripts, and separately hosted model artifacts for reviewing execution paths, output formats, and methodology. The full internal Linux benchmark, training corpus, and row-level pressure-stream data are not public, but aggregate metrics are available. The current L1 release baseline reaches **98.49% top5 any-hit**, **95.44% top5 all-covered**, and **96.44% micro recall@5** on the full Linux final gold benchmark; on K8s final gold, it reaches **100.00% top5 all-covered**.

The current result is not the product of large private logs alone. The release baseline uses an internal corpus on the order of 80,000 rows and trains in approximately **16.2 hours on a single RTX 5090**. Under the current internal source accounting, less than 1% of training data can be traced to real production or pressure telemetry. This suggests that early capability in a behavior semantic layer comes primarily from the tag system, RFC, boundary samples, and hard-negative design, not from large-scale real data alone.

On performance, SecEBL is no longer limited to offline analysis. The current RTX 5090 serving spot check reaches **5.3k unique cmdlines/s**; under highly repetitive real background traffic, a warm exact raw-event cache reaches **1.8M rows/s**; and L2 session state plus final scoring runs at approximately **151k events/s** in the in-memory path. The 7M pressure stream validates cache behavior, session aggregation, synthetic attack hits, and reviewable alert volume under large-scale real background traffic.

The conclusion is narrower: **embedding itself is not the contribution; the contribution is engineering embedding-based intent recognition into a middle semantic layer for security detection.** It allows detection systems to ask not only "did this string match a rule", but also "what behavioral intent does this event present, and can these behaviors form an attack chain". Equally important, SecEBL turns post-training miss review into a feedback loop over RFCs, gold labels, corpus coverage, and hard negatives. Model iteration is therefore not just parameter tuning; it becomes continuous convergence of security semantic boundaries.

This framing does not abandon rules, intelligence, or human review. It places them on top of a layer closer to behavioral understanding.

Core Contributions

This paper does not claim that attaching AI to an IDS is, by itself, a contribution. The claim is narrower and more technical: **intrusion detection lacks a behavior semantic layer between raw telemetry and final alerting**, and SecEBL attempts to make that layer trainable, auditable, composable, and operational.

The contributions can be summarized in four points:

1. **Redefining the detection matching unit**: moving from strings, IOCs, and fixed parameter combinations toward visible behavior-intent evidence such as `read_credential_material`, `execute_in_workload`, and `grant_cloud_privilege`.
2. **Turning "behavior understanding" into an engineering interface**: SecEBL consolidates behavior semantics into 12 behavior groups, 361 behavior labels, and an RFC specification. The model does not generate free-form judgments. It retrieves the most relevant behavior evidence from a fixed vocabulary, making outputs reviewable, measurable, and consumable by downstream systems.
3. **Turning model errors into semantic boundary convergence**: miss review is not limited to score inspection. Errors are attributed to RFC ambiguity, gold-label issues, coverage gaps, model confusion, or L2 policy gaps, and then fed back into specifications, corpus rows, hard negatives, and training strategy.
4. **Demonstrating an initial deployment profile at manageable cost**: the current release baseline is trained on an 80k-scale corpus, on a single RTX 5090, in approximately 16.2 hours. Linux final gold, K8s AuditLog, public example benchmarks, L1 serving profiles, the L2 session scorer, and the 7M pressure stream together show that the behavior semantic layer has accuracy, cross-telemetry feasibility, and an initial deployment profile.

The technical distinction in SecEBL is not embedding itself. It is the insertion of embedding-based semantic retrieval into security detection engineering: RFCs and corpus design constrain upstream behavior boundaries; rules, policies, session models, and analysts consume the downstream evidence; and interpretable intent labels connect the two.

1. Problem Definition

Security telemetry sits between two worlds.

On one side are structured fields: process name, command line, user, namespace, resource, verb, path, IP, domain, and exit code. On the other side are semi-natural-language surfaces: command arguments, filenames, log phrases, object names, request URIs, tool-specific subcommands, error messages, and audit summaries. Attackers and operators often use the same tools.

This creates several persistent detection problems:

1. **Fragile strings**: the same behavior can have many surface forms. `bash -i`, `nc -e`, `socat exec`, and `python -c` may all express shell execution or reverse-shell behavior.
2. **Reuse of legitimate tools**: LOTL behavior does not become safe because the tool is legitimate. `kubectl`, `aws`, `ssh`, `curl`, `grep`, `tar`, and `systemctl` can appear in routine operations and attack chains.
3. **Platform fragmentation**: Linux commands, K8s AuditLog, cloud audit logs, and identity logs have different syntax but may express the same security behavior, such as reading secrets, executing in workloads, modifying privileges, or establishing persistence.
4. **Weak explanation at the single-event verdict level**: a binary classifier may output malicious or benign, but detection engineers still need to know why the event is suspicious.
5. **High rule maintenance cost**: when new tools, parameters, and cloud APIs appear, allowlists, denylists, and rules can lag behind quickly.

SecEBL does not aim to replace final detection logic. It provides a better intermediate representation: normalizing visible behavior in telemetry into behavior-intent labels.

2. Design Goals

SecEBL has five design goals:

1. **Objectivity**: L1 labels describe only behavior visible in a single event. They do not encode maliciousness, authorization, success, organizational identity, or session-level intent.
2. **Composability**: behavior labels emitted from a single event can be combined by session scorers, rules, policies, or analysts.
3. **Cross-telemetry reuse**: Linux commands and K8s AuditLog use the same behavior vocabulary. Future extensions can cover cloud audit, identity events, and container events.
4. **Interpretability**: output is not an opaque score. It is ranked behavior evidence.
5. **Deployability**: L1 inference and L2 aggregation must approach throughput levels acceptable for EDR, SIEM, and streaming telemetry pipelines.

2.1 Three Claims: Foundation, Understanding, Deployability

The design can be summarized in three claims:

1. **From string rules to behavioral semantic primitives** Traditional rules and regular expressions are good at answering whether a string, path, domain, hash, or parameter appeared. SecEBL asks a question closer to detection engineering: what visible behavior appears in this event? It does not replace rules. It transfers the composability, auditability, and pipeline compatibility of rules into a behavior semantic layer. Downstream policy can still be written, but it operates on behavior evidence such as `search_credentials`, `spawn_reverse_shell`, and `query_service_health` rather than only raw strings.
2. **From keyword relatedness toward operational semantic understanding** Security telemetry is a mixture of natural-language cues and structured fields: command arguments, log phrases, filenames, resources, verbs, namespaces, and object names all carry meaning. SecEBL uses the basic capability of NLP embeddings to place raw events and behavior tag definitions in the same semantic space. It does not claim to understand organizational authorization or business intent. It does, however, capture more of the operation than keyword rules can, for example distinguishing credential search, auth log inspection, workload execution, service health queries, and cloud privilege grants.
3. **From offline benchmark to a usable detection path** A behavior semantic layer that cannot run at practical speed is only an offline analysis tool. SecEBL already has a basic serving profile: approximately 5.3k unique command inferences per second on an RTX 5090, 1.8M rows/s with a warm exact raw-event cache, and approximately 151k events/s for L2 session state plus final scoring in the in-memory path. The 7M pressure stream further validates exact-cache behavior, session aggregation, and reviewable alert volume under highly repetitive real background traffic.

The current architecture has two layers:

```
L1: event -> ranked behavior-intent labels
L2: session of L1 labels -> session risk / alerts
```

L1 is the stable core. L2 is an experimental session scorer that shows behavior labels can be consumed by downstream models, but it is not the only possible L2 implementation.

3. Technical Path: From Corpus to Real-Traffic Pressure Testing

SecEBL is not simply "training an embedding model". It is an end-to-end engineering path from security corpus design, tag system design, labeling specification, training feedback, and performance engineering to real-traffic pressure validation.

The engineering path has nine stages:

```
corpus
  -> tag system
  -> RFC labeling
  -> L1 embedding training
  -> corpus/tag optimization from misses
  -> training optimization
  -> performance optimization
  -> K8s generalization
  -> 7M pressure stream validation
```

At the system level, these stages answer six questions:

1. **Does the corpus cover real boundaries?** Coverage should not be measured by random row count. It should cover tools, parameter combinations, and operand scenarios.
2. **Can labels be trained and consumed?** Behavior should not be decomposed into fragmented axes. SecEBL returns to flat `behavior_tags[]`, where each tag represents a complete visible behavior.
3. **Can labeling be reviewed?** `tags_schema_rev20.json` and `rev20_tag_rfc.md` define boundaries so that AI systems or reviewers do not label purely by intuition.
4. **Did the model learn security semantics?** L1 does not classify maliciousness. It retrieves behavior-tag semantic text that matches the event text.
5. **Can errors drive convergence?** Miss review is attributed to RFC, gold labels, coverage, model confusion, or L2 policy, not simply to a need for "another training run".
6. **Can the system enter a detection path?** Beyond accuracy, the system must validate K8s generalization, serving throughput, exact caching, and pressure-stream alert volume.

The following sections expand these stages.

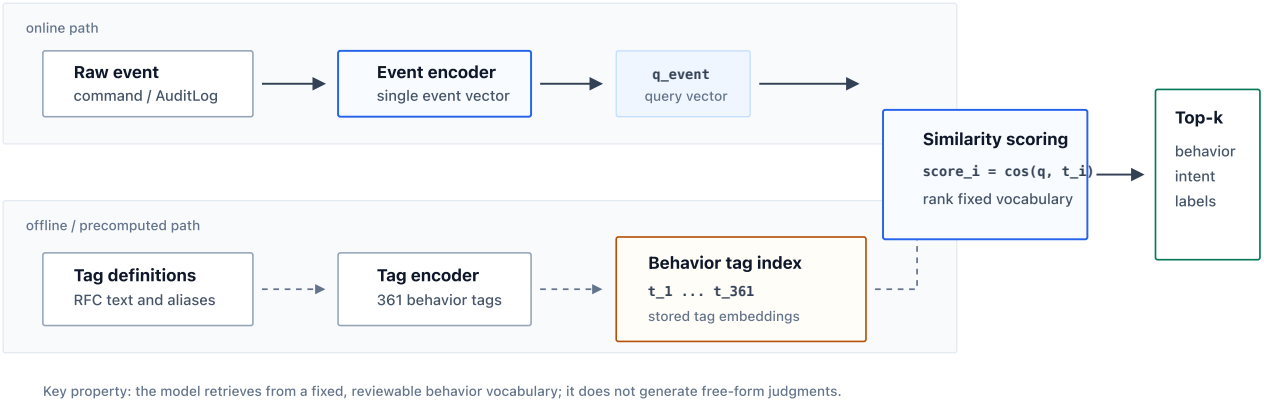
4. Background: What Embedding Does Here

Embedding can be understood as a way to place text into a vector space. A model encodes a text into a numeric vector. If two texts express similar meaning, their vectors should be close in that space. Here, semantic closeness is not merely sharing a keyword. It reflects similar behaviors, objects, and operational relationships learned during training.

In SecEBL, **embedding does not directly output "attack" or "normal". It performs a controlled semantic retrieval task**:

```
raw event text
  -> event embedding
  -> compare with behavior-tag embeddings
  -> ranked top-k behavior labels
```

Figure 2. Embedding retrieval over a fixed vocabulary



Key property: the model retrieves from a fixed, reviewable behavior vocabulary; it does not generate free-form judgments.

The system prepares two kinds of text:

1. **Event text**: for example, a Linux command line or a normalized Kubernetes AuditLog event.
2. **Tag semantic text**: for example, the definitions, boundary notes, and alias cues for `read_credential_material`, `execute_in_workload`, or `query_service_health`.

At L1 inference time, the model places the event text and 361 behavior tag definitions in the same vector space, computes similarity, and returns the closest top-k behavior labels. This is better understood as retrieving the best-matching semantic evidence from a fixed behavior vocabulary, not as generating an explanation freely.

Consider the difference from keyword rules:

```
grep "password" /etc/shadow
grep "password incorrect" /var/log/auth.log
```

Both commands contain `password`, but their security semantics differ. The value of embedding is not that it ignores keywords. It combines operation, path, object, and tag definition to determine whether the current event is closer to credential search or auth audit log inspection.

Embedding is not a complete solution. It does not know organizational authorization, asset criticality, or business context, and it does not replace IOCs, allowlists, denylists, or human review. SecEBL uses embedding because security telemetry contains many natural-language-like cues, and behavior labels can also be written as explicit natural-language definitions. The model aligns the two; RFCs, corpus review, hard negatives, and downstream policy constrain the boundaries.

From a deployment standpoint, this retrieval design also has an engineering advantage: the label set is fixed, tag embeddings can be precomputed, high-frequency raw events can be exact-cached, and L1 output remains ranked evidence rather than free-form text.

4.1 What Fine-Tuning Does Here

SecEBL does not train a large model from scratch. It fine-tunes an existing embedding base model for this task. Fine-tuning can be understood as teaching the general model security-specific boundaries. A general model may know natural-language similarity, but it does not know why `grep password /etc/shadow` and `grep "password incorrect" /var/log/auth.log` should not map to the same behavior label. Fine-tuning teaches that boundary.

The training task can be summarized as:

```
event text + correct behavior tag text
  -> pull together in embedding space

event text + confusing wrong tag text
  -> push apart in embedding space
```

Training samples are not a simple "command -> malicious/normal" classification task. They align event text with the correct behavior tag definition. The model sees the raw event and the semantic text for each behavior tag. After training, correct tags should be closer to the event, while incorrect but similar tags should be pushed away.

Hard negatives are needed at this point. Easy negatives are usually too distant; reverse shell and service health check, for example, are rarely confused. The decisive cases are high-confusion boundaries: credential search vs auth log inspection, download vs probe, archive creation vs archive listing, workload execution vs workload inspection. SecEBL uses MultipleNegativesRankingLoss and a hard-negative-aware batch sampler to repeatedly separate these adjacent semantics within the same training neighborhood.

Fine-tuning, therefore, is not about making the model memorize 80,000 commands. It teaches the model the boundaries of a security behavior vocabulary: which surface forms belong to the same behavior, and which similar-looking forms have different security meanings.

5. Evolution of the Tag System: Why SecEBL Returned from Multi-Axis to Single-Axis

SecEBL's tag system did not begin as the current flat behavior-tag design. The project tried several directions: an early single-column label design, a dual-axis design that separated behavior from object or context, and a more detailed six-axis structured schema. It eventually returned to the current single-axis flat `behavior_tags[]`.

This return to a single axis is not a return to coarse classification. It is a move from fragmented axis combinations back to complete behavioral semantic units.

5.1 Problems with the Early Single Axis

The earliest single-axis labels were simple: one command mapped to one or more labels, and training and evaluation were direct. However, early labels often mixed behavior, risk, object, context, and attack intent, making boundaries unstable.

For example:

```
cat /etc/passwd
grep password /var/log/auth.log
kubectl exec pod -- /bin/sh
curl -fsS http://127.0.0.1:8080/healthz
```

If a tag system only distinguishes broad categories such as discovery, credential, execution, and network, many important distinctions are lost. Reading an auth log is not the same as reading credential material. A health check is not the same as outbound network activity. `kubectl exec` indicates execution inside a workload, but the inner command may also express shell execution, credential reading, or process inspection.

The fundamental weakness of the early single-axis design was unstable granularity.

5.2 Why Dual-Axis and Six-Axis Designs Appeared

The motivation for multi-axis schemas was reasonable: decompose a complex command into dimensions such as operation, object, scope, platform, risk, and context. This appears to reduce label count and avoid defining a separate label for every combination.

A typical design might look like this:

```
operation = read / search / execute / modify / upload
object    = credential / config / log / workload / cloud-policy
scope     = local / remote / container / kubernetes / cloud
context   = admin / attack-like / routine / unknown
```

This is attractive as schema design, but it created practical problems in training and detection:

1. **Axes are not independent** `read + credential`, `search + credential`, `read + auth_log`, and `search + auth_log` are not simple combinations. Their security meaning depends on tool, path, arguments, search terms, and context.
2. **Label composition becomes a downstream burden** If L1 outputs fragments, L2 or rules must reconstruct the complete behavior. This propagates L1 errors into L2 and complicates explanation.
3. **Evaluation becomes difficult** Multi-axis output requires deciding whether each axis is correct, whether combinations are correct, and whether missing axes count as partial credit. It becomes difficult to answer a simple question: did the model identify the event's security behavior?
4. **Human labeling consistency decreases** Reviewers can often decide that an event is `read_credential_material`. It is harder to consistently decide how each axis should be split, which axes should appear, and which axes are only implicit.
5. **The embedding objective becomes weaker** Embedding retrieval works well when an input event is close to a complete semantic description. If labels are too fragmented, the model learns many local surface correlations rather than the complete behavior boundaries needed by detection engineering.

5.3 Current Single-Axis Tradeoff

SecEBL therefore chose flat `behavior_tags[]`, but each tag must represent a complete visible behavior, not a generic fragment.

For example:

Not split this way	Expressed this way
`read` + `credential`	`read_credential_material`
`execute` + `workload`	`execute_in_workload`
`grant` + `cloud` + `privilege`	`grant_cloud_privilege`
`search` + `suid`	`search_suid_files`
`modify` + `systemd`	`modify_systemd_unit`

The key benefit is that L1 output is itself behavior evidence that detection engineering can consume. It remains a single axis, but it is not a low-dimensional coarse category. It is a complete behavior vocabulary.

The current schema has five core constraints:

1. Labels describe visible behavior, not maliciousness.
2. Labels must be complete detection semantics, not fragments.
3. Groups are used for maintenance and reporting, not as model-output axes.

4. Multi-label output appears only when a command contains multiple independent visible operations.
5. If a more specific tag covers the behavior, a generic parent tag is not added.

This is a key engineering decision in SecEBL: sacrificing the formal elegance of a multi-axis schema for consistency across training, evaluation, explanation, and downstream consumption.

6. Corpus Quality: AI-Assisted Generation Cannot Replace Labeling Rules

SecEBL's corpus development followed a clear pattern: AI-assisted expansion is useful, but AI cannot be the semantic authority for labels. Corpus quality comes from the combination of coverage strategy, RFC rules, and human review.

6.1 Where AI Labeling Fails

Early AI-assisted generation and labeling can expand coverage quickly, but several boundary problems recur:

1. **Over-labeling on sensitive terms** `grep "password incorrect" /var/log/auth.log` contains `password`, but it is primarily reading or filtering an auth audit log. It is not equivalent to searching credential material.
2. **Ignoring negation and exclusion semantics** `grep -v password file` excludes lines containing password and should not be treated as positive credential search. `grep -L` and `find ! ...` create similar issues.
3. **Treating quoted text as executed behavior** Payloads written into cron, systemd, Dockerfile, CI config, or shell startup files are future behavior. Unless the current command also executes the payload, L1 must not label payload contents as current execution.
4. **Treating tool name as behavior** `curl` is not always download, `kubectl` is not always attack, and `ssh` is not always remote execution. The decision must consider verbs, flags, operands, and execution boundaries.
5. **Unstable read/search/modify/transfer boundaries** `cat .env`, `grep token .env`, `tar czf secrets.tgz .env`, and `curl -F file=@.env ...` are different operations, although all involve credential or sensitive material.
6. **Routine operations and attacks may look similar** Backups, restore drills, health checks, compliance evidence collection, debug tracing, and service restarts can resemble attack behavior. High-sensitivity paths or tool names alone are insufficient to label an event as malicious.

These problems show that AI can help expand samples, but without explicit rules it will often confuse surface relatedness with security semantics.

6.2 The Role of the RFC

The current label authority has two sources: `tags_schema_rev20.json` and `rev20_tag_rfc.md`. In the open-source repository, the RFC has been published as an independent 1,727-line specification. It defines the evidence model, decision procedure, tagging principles, schema boundary notes, corpus row shape, corpus change gate, and many boundary examples.

The most important principles include:

1. **visible operation**: labels must come from the currently visible operation.
2. **current execution boundary**: execution labels apply only to the stage currently executed.
3. **reference operands are usually not content reads**: a file used as configuration input, default value, or argument does not automatically imply reading its semantic contents.
4. **quoted text / search pattern is not itself execution behavior**.
5. **path/content identifiers can classify an existing operation, but cannot create an operation by themselves**.
6. **tool name is not behavioral evidence**.
7. **strong identifiers can classify targets**: names such as `.env`, `id_rsa`, `authorized_keys`, `auth.log`, `kubeconfig`, and `token` carry security meaning when used in visible read, search, transfer, archive, or

modification operations.

The RFC moves corpus labeling from model or reviewer intuition back to reviewable operational semantics. It also underpins hard negatives, boundary samples, and tool-specific review.

6.3 A Safer Pattern for AI-Assisted Corpus Generation

AI can be used extensively for corpus generation, but not as an unconstrained template generator. A safer method is coverage-driven synthesis:

1. Select a tool or behavior boundary.
2. Identify important flags, subcommands, parameter combinations, and operation modes.
3. Generate diverse parameter forms: paths, URLs, internal addresses, object names, namespaces, credential files, archive names, and output paths.
4. Cover positive, negative, and boundary cases: positive search vs negative filtering, archive listing vs creation/extraction, HEAD/spider probe vs download, ConfigMap content read vs resource enumeration.
5. Review against the RFC.
6. Pass schema checks, duplicate checks, similarity checks, and strict validation gates.

AI is useful as a sample synthesizer, not as the final semantic judge.

6.4 Single-Tool Review: Coverage Matters More Than Row Count

SecEBL corpus quality is not measured simply by row count. It depends on whether key usage surfaces of a tool are covered. The current corpus quality loop operates at the level of a single tool:

```
select tool
  -> measure flag / flag-combination / parameter-scenario gaps
  -> hand-write or AI-assisted gap rows
  -> RFC review
  -> strict validation
  -> merge into main corpus
  -> repeat
```

The loop focuses on three forms of coverage:

1. **Flag coverage**: whether important short and long flags appear.
2. **Flag-combination coverage**: whether high-value parameter combinations appear.
3. **Parameter-scenario coverage**: whether paths, URLs, secrets, config files, logs, namespaces, and cloud objects are sufficiently diverse.

This is more effective than generating another 10,000 random commands. The model needs boundaries and diversity: what the same tool means under different flags, operands, paths, resources, and contexts, and which similar-looking commands should not map to the same label.

7. Hard Negatives: Handling High-Confusion Security Boundaries

Standard embedding training pulls positive pairs closer and pushes negative pairs apart. The difficult cases in security semantics are not random negatives. They are negatives that look similar on the surface but represent different behavior.

Examples:

Anchor behavior	Hard negatives
`search_credentials`	`read_auth_audit_log`, `read_ssh_policy`, `read_credential_material`
`download_external_content`	`probe_web_application`, `query_service_health`, `connect_external_service`
`create_archive`	`list_archive_content`, `extract_archive`
`increase_file_permission`	`decrease_file_permission`, `modify_file_permission`
`execute_in_workload`	`inspect_workload`, `modify_workload`
`read_cluster_secret`	`enumerate_cluster_secrets`, `create_cluster_secret`

These are not random misclassifications. They often share a tool, path, keyword, or operation object. Random in-batch negatives are usually too easy and train the model too coarsely.

The current training uses MultipleNegativesRankingLoss, or MNRL, and a hard-negative-aware batch sampler. The core mechanism is:

1. Avoid duplicate positive labels within a batch where possible.
2. Place configured high-confusion anchors and hard negatives in the same batch.
3. Let MNRL's in-batch negative mechanism force the model to distinguish adjacent semantics.
4. Conservatively upsample curated boundary samples so that high-value boundaries receive more training exposure.

Hard negatives matter because many key detection errors occur at adjacent boundaries, not between unrelated categories.

Typical boundaries include:

1. `grep password /etc/shadow` vs `grep "password incorrect" /var/log/auth.log`
2. `grep password file` vs `grep -v password file`
3. `curl -I https://host/path` vs `curl -o /tmp/payload https://host/payload`
4. `tar tf backup.tgz` vs `tar czf exfil.tgz secrets/`
5. `kubectl get secret ...` vs `kubectl get configmap ...`
6. `chmod 700 file` vs `chmod 777 file`
7. `ssh host uptime` vs `ssh -D 1080 host`

7.1 Fine-Grained Contrast Examples

The public SecEBL release includes a small but important set of fine-grained contrast examples. These examples are not meant to inflate benchmark coverage. Their value is that they show what a behavior-intent layer can express that traditional rules struggle to express directly: the same tool, token, or object can imply different behavior depending on operation, flag, path, and context.

The examples below were scored with the public SecEBL release model. Scores are retrieval scores after the release prompt profile, and the table shows the top 3 L1 labels.

Event	Top 3 L1 tags	Interpretation
`nc -e /bin/sh 203.0.113.10 4444`	`spawn_reverse_shell` 0.811; `connect_external_service` 0.488; `spawn_bind_shell` 0.451	`-e` moves the event from ordinary network connection toward reverse-shell execution.
`nc -v 203.0.113.10 443`	`connect_external_service` 0.732; `spawn_reverse_shell` 0.503; `create_reverse_tunnel` 0.412	Connection intent ranks above shell-spawn intent even though the same tool is used.

Event	Top 3 L1 tags	Interpretation
`cat /root/install.log`	`read_business_log` 0.641; `read_system_log` 0.431; `read_workload_logs` 0.385	The model treats the operation as log reading.
`cat /root/install.conf`	`read_infrastructure_config` 0.620; `read_system_config` 0.612; `read_kernel_parameter` 0.336	A similar read operation shifts toward configuration semantics because the object changes.
`grep -a "password" /var/log/auth.log`	`read_auth_audit_log` 0.675; `search_credentials` 0.609; `read_credential_material` 0.507	Both auth-log reading and credential-search evidence are visible; final risk should be decided downstream.
`grep -v "DEBUG" /var/log/app.log`	`read_business_log` 0.824; `delete_business_log` 0.515; `read_business_config` 0.315	A negative filter remains primarily log-read behavior, not destructive or credential behavior.
`grep "password" /etc/shadow`	`search_credentials` 0.752; `read_credential_material` 0.698; `crack_credential_material` 0.296	The same `grep` and `password` surface now maps to credential-access semantics because the source object is credential material.
`grep "password incorrect" /var/log/auth.log`	`read_auth_audit_log` 0.731; `search_credentials` 0.548; `read_credential_material` 0.452	The phrase contains `password`, but the dominant behavior is auth-log inspection rather than credential-material access.

This is an important threshold result for SecEBL. It does not mean that a small embedding model has complete security understanding, and it does not remove the need for rules, policies, or analyst review. It does show that a small fine-tuned embedding labeler can learn part of the behavior boundary that rules usually express only through long exception lists: the model can separate connection from shell execution, log reads from config reads, positive credential search from auth-log inspection, and negative filtering from content search. In practical detection engineering terms, this moves part of detection from handcrafted string-condition expression toward direct behavior-evidence extraction. The system can produce behavior evidence before final risk scoring, instead of forcing every distinction into brittle string patterns.

This is why SecEBL moved from expanding the corpus to refining boundaries. Once broad coverage is in place, the decisive factor is not ordinary samples but high-confusion boundaries.

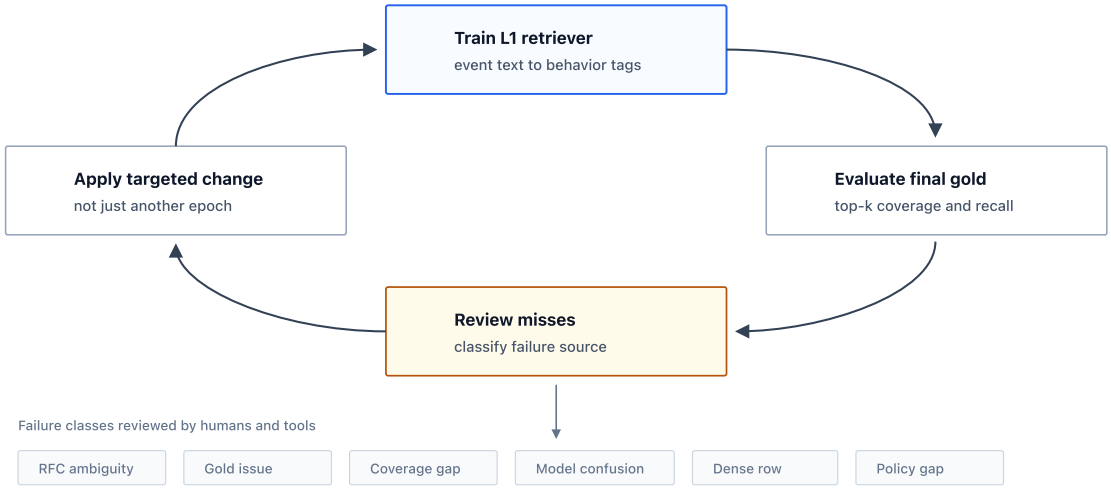
8. Miss-Driven Training Loop: From Errors to Semantic Boundary Convergence

If the training section only discussed epoch count, batch size, GPU, and loss function, its technical value would be limited. Those details are necessary engineering facts, but they are not the research contribution. The important part is the loop after training: **model errors are not merely counted; they are used to discover security semantic boundary problems.**

The current loop can be summarized as:

```
train L1 embedding retriever
  -> evaluate final-gold top-k
  -> inspect miss / top1-to-top5 gap rows
  -> classify the failure source
  -> update RFC, gold label, corpus coverage, hard negatives, or upsampling
  -> retrain and compare the same metric family
```

Figure 3. Miss-driven semantic boundary loop



The core value of this process is that **model misses become a boundary discovery mechanism.** When the model fails to place a gold tag in top5, or when top1 hits an adjacent but incomplete label, the reviewer does not immediately attribute the error to insufficient model size. The reviewer first classifies the failure source.

Miss type	Typical symptom	Follow-up action
RFC ambiguity	A reviewer can justify two labels, but the specification does not define the boundary	Update RFC core text or boundary examples
Gold-label issue	The gold label violates visible-operation or specificity principles	Correct the gold or benchmark label
Coverage gap	Corpus lacks a tool, parameter combination, or operand scenario	Add targeted rows after single-tool review
Model confusion	Top-k contains adjacent semantics, such as read/search, probe/download, inspect/modify	Add hard-negative pairs or hard-negative batch configuration
Dense-row miss	A row contains multiple independent behaviors and some gold tags do not enter top5	Add multi-operation samples and preserve ranked evidence rather than collapsing to one label
Runtime policy gap	L1 labels are correct, but the session scorer does not interpret them under environment context	Adjust L2 features, thresholds, or policy rather than changing L1 labels

This classification prevents every error from being treated as "train again". SecEBL's semantic quality comes from the convergence of three paths:

1. **Specification convergence**: misses expose RFC boundaries that are not clear enough. The boundary is written back into the RFC rather than remaining in reviewer memory.
2. **Corpus convergence**: misses point to specific tools, parameters, paths, resources, namespaces, cloud objects, or log phrases. The response is targeted corpus rows, not random generation.
3. **Training convergence**: when the RFC and gold label are clear but the model still confuses adjacent semantics, the case is added to hard negatives, batch sampling, or boundary upsampling.

A typical example is the credential/log boundary:

```
grep "password" /etc/shadow
grep "password incorrect" /var/log/auth.log
grep -v "password" /var/log/app.log
```

All three commands share `grep` and `password`, but they correspond to credential search, auth audit log inspection, and negative filtering. Early models and AI labels can easily pull them into the same semantic region. Miss review separates the problem: the RFC must clarify the relationship between search pattern and source file; the corpus must cover positive search, log phrases, and exclusion filtering; and hard negatives must force `search_credentials`, `read_auth_audit_log`, and `read_business_log` into the same training neighborhood.

Kubernetes provides another example:

```
k8s_audit verb=get resource=secrets ...
k8s_audit verb=list resource=secrets ...
k8s_audit verb=create resource=pods subresource=exec ...
```

These events share the `k8s_audit` format and similar fields, but their semantics may be reading secret value, enumerating secret metadata, or executing inside a workload. Miss review can determine whether the issue is an unclear K8s RFC rule, missing normalized AuditLog corpus forms, or a model confusion between `resource=secrets` metadata and data access.

Experimentally, the value of the loop appears in metric movement, but it should not be attributed to a single training parameter. From an earlier final-gold baseline to the current release baseline, Linux top5 all-covered improved from 89.80% to 95.44%, micro recall@5 from 91.62% to 96.44%, and dynamic exact from 70.40% to 87.32%. This is not a conclusion about more epochs alone. It is the combined effect of data, RFCs, gold review, prompt and semantic text, hard negatives, boundary upsampling, and training configuration.

The training section is therefore not primarily about training an embedding model. It is about the following:

1. Model evaluation can expose security semantic boundaries.
2. Miss review attributes errors to RFC, gold labels, coverage, model confusion, or L2 policy.
3. Corpus expansion moves from quantity-driven to boundary-driven.
4. Hard negatives teach the model different behaviors under similar text.
5. A consistent final-gold metric family makes iterations comparable rather than self-validating on different benchmarks.

This distinguishes SecEBL from a typical AI IDS proof of concept: training becomes a joint convergence loop for the tag system and detection semantics.

9. Why Semantic Embedding Fits Security Telemetry

Security telemetry is not a purely structured table. It naturally mixes structured fields with natural-language-like cues:

1. Command names and subcommands: `kubectl exec`, `aws iam attach-user-policy`
2. Parameter names: `--policy-arn`, `--secret-id`, `--data-urlencode`
3. Paths and filenames: `.env`, `authorized_keys`, `auth.log`, `kubeconfig`
4. Object names: `ClusterRoleBinding`, `payment-api-token`
5. Log phrases: `Failed password`, `password incorrect`
6. API verbs and resources: `verb=create resource=pods subresource=exec`
7. Tool ecosystem semantics: Terraform plan/apply/destroy, Vault read/write, Docker run/exec/cp

These fields are not ordinary strings. They often directly express operation object, operation mode, and security boundary. Traditional rules can match keywords, but they have difficulty understanding the role of those words in the current event. SecEBL uses embedding because both raw events and behavior tag definitions can be written as semantic text, and the model can learn the relationship between them.

9.1 Semantics Are Not Keywords

```
kubectl -n prod get secret payment-api-token -o jsonpath='{.data.token}' | base64 -d
kubectl -n prod get pods -o wide
```

Both begin with `kubectl get`, but the first represents secret read and decode, while the second represents workload or resource enumeration. A semantic model can use `secret`, `token`, `jsonpath`, `base64 -d`, and resource type together rather than relying only on the tool name.

9.2 Tag Semantic Text Defines Retrieval Boundaries

SecEBL embeds not only commands but also the semantic text of behavior tags into the same space. The training objective is to bring command events close to the correct behavior definitions.

Therefore, tag semantic text cannot be only the tag name. Current training uses compact semantic text and adds RFC-derived aliases for high-confusion labels, such as `sudoers`, `authorized_keys`, `ClusterRoleBinding`, `Vault`, `tcpdump`, and `pcap`. These aliases are not new rules. They expose stable semantic cues from the RFC to the model.

This makes L1 retrieval closer to security semantic matching rather than string matching on tag names.

10. Direct Comparison with Denylists, Allowlists, and Traditional Rules

SecEBL does not improve detection by matching known IOCs faster. It shifts the detection unit from strings to behavioral intent.

Dimension	Denylists / allowlists / rules	SecEBL behavior-intent layer
Matching unit	Strings, hashes, IPs, domains, paths, tool names, fixed parameter combinations	Visible behavior, such as credential read, workload execution, cloud privilege grant
Generalization	Strong on known forms, weak on variants	More stable across different surface forms of the same behavior
LOTL scenarios	Legitimate tools can make rules too broad or too narrow	Tool legitimacy does not imply harmless behavior; behavior evidence is extracted first
Cross-platform	Linux, K8s, and cloud rules are usually written separately	Different telemetry sources can map to the same behavior vocabulary
Interpretability	Tells you which rule matched	Tells you which behavioral intents are visible in the event
Composition	Attack-chain rules must be written manually	L1 tags can feed L2 session scorers or policy
Maintenance	New syntax requires new rules	New syntax can be learned through targeted corpus rows, tool review, and hard negatives
Risk	Precise IOCs are strong, but their semantic scope is narrow	Broader semantics, but probabilistic outputs and training boundaries must be managed

Direct examples:

Scenario	Why traditional rules struggle	SecEBL representation
reverse shell	Must cover `nc`, `bash`, `socat`, `python`, `perl`, and other forms	`spawn_reverse_shell` or related execution/network tags
credential access	`password` may indicate secret search or only an auth log phrase	`search_credentials` vs `read_auth_audit_log`
Kubernetes exec	Looking only at `kubectl` or `pods/exec` is insufficient; the inner command matters	`execute_in_workload` plus inner Linux behavior
cloud privilege change	Each cloud provider and CLI has different syntax	`grant_cloud_privilege`, `modify_cloud_identity_policy`
benign maintenance	`systemctl`, `pg_dump`, and `kubectl rollout` may be normal or abnormal	L1 provides behavior evidence; L2/policy combines session context

The capability gain has three layers:

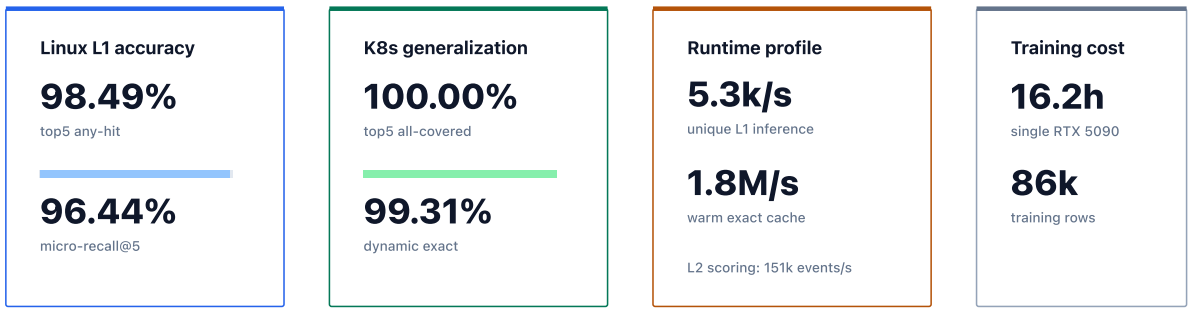
1. **Single-event generalization**: from fixed strings to behavior semantic retrieval.
2. **Multi-event composition**: from single hits to session-level intent chains.
3. **Cross-source reuse**: from rewriting rules per platform to sharing a behavior vocabulary.

SecEBL does not reject traditional rules. IOCs, allowlists, and denylists remain effective for known threats and strongly constrained assets. SecEBL provides a behavior semantic layer alongside those mechanisms, especially for variants, LOTL behavior, cross-platform semantics, and interpretability.

11. Current Experimental Results: Accuracy, K8s Generalization, and Performance

This section answers three questions: whether SecEBL is accurate, whether it generalizes across telemetry types, and whether it can run at practical speed.

Figure 4. Current evidence at a glance



> **Key results**: **Linux L1 top5 any-hit 98.49% / micro-recall@5 96.44%**, **K8s top5 all-covered 100.00%**, and **warm exact raw-event cache 1.8M rows/s**. The tables provide audit context for these numbers.

11.1 Accuracy: L1 Is the Primary Result, L2 Is Experimental Validation

SecEBL's primary result is L1 behavior-label retrieval. Given a security event, the model must retrieve the behavior labels closest to the visible behavior from a fixed vocabulary of 361 labels. This evaluation looks only at final gold top-k. It does not include L2, threshold tuning, or session policy.

The metrics below use the internal final benchmark. The open-source repository provides a publishable example subset for reviewing code paths, RFCs, schema, and output format; it is not the complete internal evaluation set.

Dataset	Rows	Sessions	Rows with gold	Gold tag instances	Unique labels
Linux final benchmark	12,594	663	11,889	17,287	361
K8s final benchmark	144	46	144	163	27

Current L1 results:

Dataset	Rows with gold	Gold tag instances	top1 any-hit	top5 any-hit	top5 all-covered	micro-recall@5	dynamic exact
Linux	11,889	17,287	92.54%	**98.49%**	**95.44%**	**96.44%**	87.32%
K8s	144	163	99.31%	**100.00%**	**100.00%**	**100.00%**	99.31%
Combined	12,033	17,450	92.62%	**98.50%**	**95.50%**	**96.47%**	87.47%

On Linux, top5 contains at least one gold tag for 98.49% of rows and covers all gold tags for 95.44% of rows. Calculated by tag instance, micro recall@5 is 96.44%. Dynamic exact is stricter: it takes as many predictions as the number of gold tags and requires an exact match, which makes dense multi-label rows harder. The current Linux dynamic exact miss count is 1,507 rows; top5 fails to cover all gold tags in 542 rows; and 180 rows have no gold hit in top5. The main miss clusters are remote authentication, temporary or hidden staging, infrastructure-service distinction, service metrics, and dense multi-label rows. K8s final gold has no top5 miss.

L2 session scorer: Can behavior labels be consumed by downstream detection?

The current L2 is an experimental fitted scorer. It uses only cached L1 semantic output and does not use raw command text, user name, host name, or session id as runtime features.

L2 is not the core model contribution of this paper, and it is not a production IDS classifier. Its value is to show that more than 300 behavior-intent tags can be consumed downstream, not only interpreted at the single-event level.

Check	Scale	Result
OOF validation	5,747 sessions	Accuracy 99.39%; attack precision 96.44%; attack recall 95.31%; normal recall 99.72%; TP / TN / FP / FN = 406 / 5,306 / 15 / 20
Internal final session fit-check	663 sessions	365 TP / 298 TN / 0 FP / 0 FN

> **Boundary note**: L2 is useful because it shows behavior-intent tags can be consumed downstream; **the final-session 100% result should only be read as a fit-check**. It does not replace OOF validation and does not represent production IDS accuracy.

The current L2 may still overfit the existing benchmark, pressure positives, hard negatives, and threshold calibration. A more mature direction is to introduce entity semantics, temporal sequence modeling, and real behavior-chain modeling on top of the behavior labels.

11.2 K8s Generalization: One Behavior Vocabulary Across Different Telemetry

The K8s result answers a separate question: whether SecEBL is only a Linux shell command classifier, or whether it can serve as a cross-telemetry behavior semantic layer.

Kubernetes AuditLog does not have conventional command-line syntax, but it includes fields such as verb, apiGroup, resource, subresource, namespace, name, userAgent, sourceIP, and requestObject. These fields also carry security semantics: `pods/exec` is workload execution, `secrets` data access is secret read, RBAC mutation is privilege or policy change, and rollout/status/log/watch are often operational or observational behavior.

SecEBL does not define a separate rule set for K8s. It normalizes AuditLog into semantic event text and maps it to the same 361-label behavior vocabulary. This allows Linux `kubectl exec`, K8s AuditLog `create pods/exec`, and future workload telemetry to map to the same behavior class.

The current K8s release data is small, but it supports the interface hypothesis:

Item	Value
K8s training corpus rows	1,008
K8s final examples	144 rows / 46 sessions
Gold tag instances	163
Unique behavior labels	27
top5 all-covered	**100.00%**
micro-recall@5	**100.00%**
dynamic exact	**99.31%**

The K8s result should be described as a cross-telemetry feasibility signal, not as complete Kubernetes IDS coverage. The key conclusion is that the same behavior vocabulary can carry both Linux command and K8s AuditLog semantics. SecEBL is not intended to be a Linux-only classifier; it is intended to normalize different security event sources into a reusable behavior-intent layer.

11.3 Performance: Deployment Feasibility

Accuracy and generalization show that the model is useful. Performance determines whether it can enter EDR, SIEM, or near-real-time detection pipelines. SecEBL now has a basic serving profile, not only an offline benchmark.

Path	Result	Meaning
L1 no-cache unique inference, RTX 5090	**5,308.72 unique cmdlines/s**	Cold path for deduplicated new events
Warm exact raw-event cache lookup	**1,817,462.76 rows/s**	Main-path throughput under repetitive traffic
L2 session state + final scoring	**mean 151,169 events/s**, best 159,695 events/s	Session aggregation is not the primary bottleneck

The 7M pressure stream validates runtime behavior and alert volume at real background traffic scale. It is not used to claim precision or recall, because it does not have row-level or session-level gold labels. Its value is in throughput, cache benefit, session aggregation, synthetic attack hits, and alert-volume review.

Pressure stream metric	Value
Rows	6,286,568

Pressure stream metric	Value
Unique raw cmdlines	486,584
Raw command repeat rate	**92.2599%**
Sessions	102,117
Alert sessions	61
Synthetic alert sessions	60
Reviewed real alert sessions	1

These results show two points. First, real background traffic is highly repetitive, so exact raw-event caching is not a minor optimization; it is a deployment prerequisite. Second, the calibrated L2 did not produce uncontrolled noise on the 7M background stream, and the final alert volume remained reviewable. This is not equivalent to production IDS accuracy.

12. Performance Implementation Details: Why Exact Cache Is Critical

The previous section gives the performance results. This section explains why those results are possible. For SecEBL to enter EDR, SIEM, or near-real-time detection pipelines, performance is not an add-on; it is a core design constraint.

L1 embedding inference is inherently heavier than string rules. Without optimization, it is difficult to use in high-throughput telemetry pipelines. The current implementation centers on four techniques:

1. FP16 inference.
2. SDPA attention.
3. Keeping command and tag embeddings on GPU and performing GPU top-k directly, returning only top-k results.
4. Exact raw-event cache to avoid repeated embedding of identical commands.

The exact raw-event cache has the most deployment significance. In the 7M pressure stream, the raw command repeat rate is 92.2599%, showing that real background traffic contains many repeated surface forms. The cache key is the exact raw event string. Cache hits reuse the same L1 top-k result. There is no fuzzy matching or approximate reuse, so the cache does not alter L1 semantics.

L2 is lighter and suitable as a session state layer. A practical high-throughput architecture is to let L1 handle cacheable semantic parsing, while L2 or downstream policies handle lightweight aggregation, thresholds, entities, and sequence logic. Together, these choices allow SecEBL to enter detection engineering paths rather than remain limited to offline evaluation.

13. Training Cost and Scalability

SecEBL's current capability is not built from massive real telemetry or a very large training set. The current release baseline uses an 80k-scale training corpus, a single RTX 5090, and approximately 16.2 hours of training time. This shows that the early cost profile of the approach is manageable.

Item	Value
Training hardware	1 x NVIDIA GeForce RTX 5090, 32GB VRAM
Base model	Alibaba-NLP/gte-modernbert-base
Training rows	**86,285 internal corpus rows**

Item	Value
Non-empty training observations	82,895
Effective command/tag pairs	118,858
Epochs / batch size	128 / 112
Precision	fp32
Training runtime	**58,291 seconds, about 16.2 hours**

Under the current internal source accounting, less than 1% of training data can be traced to real production or pressure telemetry. Most training signal comes from RFC-constrained synthetic samples, hand-written samples, reviewed examples, tool-specific coverage, and hard negatives. The conclusion should remain limited: an 80k-scale corpus and single-GPU training demonstrate cost efficiency and scaling potential; they do not prove coverage of all enterprise environments, platforms, or attack techniques.

14. Future Work

SecEBL is currently closer to a behavior semantic infrastructure layer than a complete intrusion detection system. The next important direction is not to keep increasing generic model metrics, but to turn the SecEBL RFC, miss review, hard negatives, corpus completion, and retraining loop into an AI-assisted detection engineering loop that can run inside enterprise intrusion detection settings.

1. **Enterprise semantic loop for intrusion detection** A general model cannot directly decide whether a behavior inside an enterprise is abnormal. The same `kubectl exec`, secret read, RBAC modification, bulk log archive, database export, or internal object-storage access may have different meanings in deployment, incident response, backup, audit, or an attack chain. What matters for intrusion detection is not generic business understanding, but detection-relevant context: asset criticality, identity roles, change windows, routine operational paths, incident-response exceptions, service dependencies, and historical alert-handling conclusions.

   The SecEBL loop can provide an auxiliary loop for enterprise detection engineering. L1 and L2 produce behavior labels, misses, false positives/negatives, and high-confusion samples. In a controlled workflow, AI assistance can cluster alerts, summarize relevant runbooks, tickets, CMDB records, and deployment records, and propose RFC boundary updates, hard negatives, normal operations samples, and attack simulation samples. Security engineers review the output before it enters training sets, evaluation sets, and detection policies. This is an intrusion-detection semantic loop, not a general business knowledge base.

   The most valuable asset in this loop is the organization's accumulated understanding of its own business operations: what each service normally does, which identities operate it, which maintenance paths are expected, which exceptions are legitimate, and which behavior chains are never normal. In the AI era, this knowledge can become a controlled data flywheel. Each reviewed alert, miss, false positive, runbook, incident note, and change record can improve local behavior boundaries, local hard negatives, and local session policies. The general SecEBL vocabulary remains portable, while the enterprise-specific semantic layer becomes more accurate precisely because it learns the organization's real operating model.

   The boundary of this direction should remain explicit. SecEBL does not automatically decide authorization relationships or final business risk. It turns repeated detection boundaries in enterprise context into reviewable label specifications, samples, and policy inputs. The long-term goal is to reduce the maintenance cost of rule patches and false-positive exceptions, and to make the behavior semantic layer better adapted to the organization's security operations environment.
2. **Entity analysis** L1 currently answers what behavior appears in an event, but it does not systematically answer whom the behavior acts on. Future work can extract and normalize entities such as principal, host, process, container, pod, namespace, resource, file path, IP, domain, credential object, cloud identity, and service account, then connect behavior labels to an entity graph. This would let downstream systems ask more detection-oriented

questions: whether the same identity enumerates secrets and then enters a workload, whether the same host stages and then exfiltrates data, or whether the same service account performs RBAC mutation and pod exec in a short period.

3. **True behavior sequence detection** The current L2 mainly uses session-level aggregate features. It shows that behavior labels are separable, but it is not a true behavior sequence model. Future versions can feed L1 top-k tags, retrieval scores, time gaps, entity changes, and operation order into sequence or graph-sequence models, so the system can learn distinctions such as credential access -> staging -> archive -> egress, discovery -> privilege change -> workload exec, and health check -> config read -> benign remediation.
4. **LOTL-like hard negatives** The difficult part of LOTL scenarios is not whether the tool is suspicious, but whether the combination and context of legitimate tools is abnormal. Future work should systematically expand normal operations, incident response, backup, debug, CI/CD, SRE automation, cloud administration, and Kubernetes controller behavior as high-quality hard negatives. The goal is not to make the model less sensitive, but to teach it the boundaries among normal high-privilege operations, gray LOTL-like operations, and malicious sessions.
5. **Cross-environment calibration** The current L2 threshold comes from the existing benchmark and pressure stream. Real environments differ in tool habits, asset roles, automation scripts, noise distribution, and alert tolerance. Future L2 designs should be calibratable by environment: the L1 semantic vocabulary remains stable, while L2 policy, sequence models, or entity graphs are trained or tuned for each organization.
6. **More telemetry types** K8s shows that the same behavior vocabulary can extend beyond Linux command lines, but it remains an early signal. Future versions can extend to cloud audit, identity provider logs, container runtime events, EDR process telemetry, network metadata, and secret-management audit logs. The key issue is not to reinvent rules per platform, but to normalize different telemetry types into the same behavior semantic layer.

All of these directions depend on the same premise: L1 behavior-intent tags must remain objective, interpretable, and reviewable. Only when L1 is stable can entities, sequences, and L2 policy avoid amplifying model errors into opaque alerts.

Author Bio

Yue (Will) Chen is a security architect and senior security engineer. He currently serves as Head of Security and Compliance at an international mobile game developer and publisher. His work spans detection and response, intrusion defense, cloud workload security, Linux host security, security product development, and large-scale security architecture. He has more than nine years of hands-on security engineering and architecture experience across internet, gaming, financial, and global infrastructure environments, including security program design, detection systems, agents, rule engines, data pipelines, incident response, and complex intrusion investigations.

One of the author's most important engineering backgrounds is host and cloud workload security in the ByteDance / TikTok ecosystem. He was responsible for Elkeid's architecture design and intrusion detection strategy. Elkeid is ByteDance's open-source Cloud Workload Protection Platform, covering servers, containers, Kubernetes, and serverless environments, and has served more than 2M servers in production. The author implemented much of Elkeid's Linux kernel driver, real-time detection and rule engine, and core detection strategy, with emphasis on performance, stability, detection quality, traceability, and operational usability. In ByteDance's 2022 internal adversarial exercise, the related detection capability reached a 100% detection rate. He also led Elkeid's open-source release and commercialization work; the project exceeded 2K GitHub stars in its first open-source year and led to commercial contracts and community deployments.

Another engineering line directly related to SecEBL is AgentSmith / AgentSmith-HUB. AgentSmith-HUB is a high-performance security data pipeline and detection platform led by the author. It includes real-time XML rules, a CEP engine, alert enrichment, and LLM agents. These systems are not about isolated model scores. They focus on how security telemetry enters high-throughput data pipelines, how it is consumed by rules and behavior engines, and how it supports alert enrichment, automated investigation, and detection engineering feedback loops. SecEBL continues this line of engineering: convert security events into interpretable behavior semantics first, then pass them to rules, session models, policies, agent workflows, or analysts.

The author has also led infrastructure and security work at a globally recognized gaming company, including self-built IDC, private cloud, KMS, server/container security, and infrastructure compliance architecture. He has also built NIDS, database auditing, anomaly detection, asset graphs, and SQL behavior auditing capabilities in a multinational financial group environment. These experiences make this paper a discussion of behavior-intent recognition from the perspective of real security products, production telemetry, detection engineering, performance constraints, and security operations loops, rather than from a purely model-centric view of IDS.

The author's public work includes the Black Hat 2023 talk "Elkeid - Open-sourced Cloud Workload Protection Platform", the GOTC 2023 talk on production multi-workload security practices, CISSP certification, 19 CVEs, multiple patents related to intrusion detection and anomaly detection, and open-source projects such as AgentSmith-HIDS and KAP. Elkeid and AgentSmith-HUB are representative engineering achievements in host/cloud workload security, security data pipelines, and detection engineering. SecEBL is a further research-oriented expression of the question built on those experiences: how can a security system understand behavioral intent?

Appendix: Evidence Entry Points

SecEBL open-source code repository: `https://github.com/EBWi11/SecEBL`

SecEBL model repository: `https://huggingface.co/willchen0011/SecEBL`